OWNER GUIDE

Mohammed — Security Lab

Delivery date: 20 October 2026. This guide describes the files and controls actually included. The website is a static React application; the editor is a local, in-memory utility, not a remotely authenticated CMS.

1. What the project contains

- Homepage, project collection and case studies, research collection and articles, achievements, about, contact, and branded 404.
- Reusable navigation, cards, buttons, filters, copy controls, Markdown rendering, code highlighting, table of contents, and responsive layouts.
- `src/content/site.json`: all owner-editable profile, SEO, theme, navigation, skills/interests, project, article, and achievement records.
- `src/content/schema.ts`: validation rules and inferred TypeScript models.
- `src/content/public.json`: generated, publication-filtered content. Never edit it manually.
- `src/pages/Admin.tsx`: local editing workspace, excluded from production bundles.
- `.github/workflows/deploy.yml`: build/test and GitHub Pages deployment.
- `tests/`, `scripts/`, `docs/`: checks, build tools, security notes, and measured results.

The three published project examples and two editorial articles are samples, not claims about completed work. One non-sensitive example article is a draft to demonstrate production exclusion. The supplied real achievement is TechWiz 6, 2025, Ethical Codebreaking, Team SabaSec, team leader, Special Jury Mention. No rank, medal, certificate, or personal contribution details are invented.

2. Selected architecture

React + strict TypeScript + Vite. Content is local JSON, validated with Zod. URL fragments (`#/projects/id`) provide static-host-safe navigation. React Router is unnecessary for this scope. Markdown is rendered without raw HTML. Article/code rendering is lazy-loaded. Fonts are self-hosted Inter Variable and JetBrains Mono.

GSAP context cleanup handles entrances/reveals under Strict Mode. Lenis React uses one GSAP ticker, `autoRaf: false`, and ScrollTrigger updates. Touch scrolling remains native. Reduced-motion preference disables smooth scrolling and motion. SpotlightCard is a simplified React Bits adaptation; the hero light is a 21st.dev/Aceternity-inspired SVG adaptation. Licensing is documented in `docs/THIRD_PARTY_NOTICES.md`.

No database, API service, analytics, tracking, cloud account, or paid plugin is required.

3. Install prerequisites

Install Node.js **22 LTS**, including npm, from the official Node.js installer. Git is needed for repository-based publishing. Confirm:

```
node --version
npm --version
git --version
```

The project was built with Node 20.20.2 and npm 10.8.2; Vite requires Node 20.19+ or a compatible newer version. CI uses Node 22.

4. Install dependencies

Extract the ZIP. Open a terminal in the `portfolio` directory (where `package.json` is):

```
npm ci
```

Use the supplied lockfile. Do not use `--force` or `--legacy-peer-deps` to bypass compatibility errors. No `.env` is required. `.env.example` contains comments only, not secrets.

5. Run the development server

```
npm run dev
```

Open the address printed by Vite, normally `http://localhost:5173/`. For a private local bind:

```
npm run dev -- --host 127.0.0.1
```

The default binds only localhost. Arena's proxy preview requires the explicit command `npm run dev -- --host 0.0.0.0`. Do not expose development servers to untrusted networks. Stop with Ctrl+C. A port already in use may cause Vite to choose another port.

After editing `src/content/site.json`, run `npm run validate` or restart the development server to regenerate public content. The local editor does not auto-sync source JSON changes into an already-open form; refresh the editor after replacing the file.

6. Build the production site

```
npm run build
npm run check:production
```

The build validates content, generates publication-filtered JSON, runs TypeScript, bundles the site into `dist/`, and generates HTML metadata, `404.html`, robots, and sitemap files. `check:production` checks that draft IDs and editor implementation text are absent from bundles. Validation failure stops the build. No source maps are published by default.

7. Preview production

```
npm run preview
```

Open the printed URL, usually `http://localhost:4173/`. This serves the build, not current unsaved source edits. Rebuild after changing content. The production control-center route explains that editing is local; it has no editor or export button.

8. Create and configure a GitHub repository

Create a repository in your GitHub account. Keep it private until you have reviewed all content, including drafts and history; confirm whether your account plan permits Pages for a private repository. From the project directory:

```
git init
git add .
git commit -m "Initial Security Lab portfolio"
git branch -M main
git remote add origin https://github.com/YOUR_ACCOUNT/YOUR_REPOSITORY.git
git push -u origin main
```

Replace the uppercase placeholders. Authenticate with your own secure Git/GitHub workflow, not a token in source files. Check `git status` before every commit. `node_modules`, build output, `.env` files, generated public content, and test browser session files are ignored. Only `dist/` is uploaded to Pages by the workflow. Do not upload the ZIP itself to the public website.

9. Deploy to GitHub Pages

In the repository, select **Settings → Pages → Build and deployment → Source: GitHub Actions**. The workflow runs on pushes to `main` or manual dispatch. It installs from the lockfile, lints, tests, builds, checks production isolation, and deploys the Pages artifact using short-lived GitHub Actions identity. No personal access token is needed.

Wait for the Actions run to succeed. Visit the deployment URL shown by GitHub. This delivery has not connected to your account, created a repository, or published a remote site. Review GitHub's current Actions policies if a workflow is blocked.

10. Configure the repository base path

A project site such as `https://YOUR_ACCOUNT.github.io/YOUR_REPOSITORY/` needs `/YOUR_REPOSITORY/`. The workflow derives it from the repository name. For a user site (`YOUR_ACCOUNT.github.io`) or a custom domain, add the **repository variable** (not a secret) `SITE_BASE` with value `/` in Settings → Secrets and variables → Actions → Variables.

For a local project-path test on macOS/Linux:

```
SITE_BASE=/YOUR_REPOSITORY/ npm run build
SITE_BASE=/YOUR_REPOSITORY/ npm run preview
```

For PowerShell:

```
$env:SITE_BASE="/YOUR_REPOSITORY/"
npm run build
npm run preview
```

Use the same base for build and preview. A base-path preview started with `/` cannot correctly serve a build intended for a repository prefix. Remove the environment variable or set `/` when returning to root hosting. Share links with fragments, for example `/YOUR_REPOSITORY/#/projects/web-assessment-notebook`.

Refreshes work because fragments are not sent to the server. Unknown fragments render 404. Arbitrary clean nested paths are not the site's routing strategy; don't publish those links.

11. Add a custom domain

Choose a domain you control. Configure it under repository Settings → Pages → Custom domain, then follow GitHub's current DNS instructions for your chosen apex/subdomain. Verify domain ownership to reduce takeover risk. Set `SITE_BASE=/` as above. Put the exact approved hostname in `public/CNAME` if you want the workflow artifact to carry it. Do not add a placeholder CNAME.

Set `seo.siteUrl` in `src/content/site.json` to the final HTTPS root URL, including repository prefix if applicable. Rebuild and redeploy. Remove a custom domain from GitHub and DNS safely when retiring the site; do not leave dangling records.

12. Enable HTTPS

After DNS and certificate provisioning complete, enable **Enforce HTTPS** in Pages settings. Verify the public site, icons, fonts, and résumé work over HTTPS with no mixed content. GitHub controls HTTP headers; the application does not claim to configure HSTS, CSP headers, or server policies on Pages. If you require custom security headers, use a host/proxy that supports them.

13. Update personal information

Edit `profile` in `src/content/site.json`, or open `#/control-center/` during development and choose **Profile**. Change `name`, `headline`, `intro`, `bio`, `focus`, and `goals`. Keep statements evidence-based. The initial text describes interests and development, not verified employment or senior consulting experience.

Settings panels use labeled, validated JSON text areas. Save in memory, then export and replace the content file as described in section 25. Do not edit ordinary content in JSX. The `homepage` object controls the hero heading, technical-focus records, workflow steps, leadership/contact copy, selected achievement ID, and visibility of optional homepage sections. Edit it through **Theme & settings** or directly in source JSON. Layout-level labels and page compositions remain in `src/pages/Public.tsx`.

14. Add and edit projects

In the local editor choose **Projects → Add record**. Use a unique lowercase hyphenated ID; it becomes the URL. Fill in title, summary, category, tags, optional type/date/repository, sample flag, featured flag, and publication status. Categories are free-form, so Web Security, Penetration Testing, Security Research, CTF, Automation and Tooling, Defensive Security, and Other are all supported.

Add sections with headings and Markdown bodies. Suggested headings: Overview, Objectives, Scope and authorization, Environment and assumptions, Methodology, Tools and technologies, Findings, Evidence, Risk analysis, Remediation, Validation, Lessons learned. Empty bodies are hidden. Add HTTPS references and optional screenshots. Image paths may be `images/your-file.webp` or `documents/...`; place the actual assets under `public/`. Do not prefix a local path with `/`. Provide alternative text and captions. Cover images have a URL and required alternative text. Section images also require non-empty alternative text. Declared local résumé, cover, and section image links for visible content are checked for file existence during validation; missing files stop the build.

Use ISO dates (`2026-10-20`) only when a publication/completion date is actually known. Invalid syntax and impossible calendar dates fail validation. A missing date is omitted. External links must use HTTPS. Do not put repository write tokens in a link.

Save, export, replace the source file, validate, review, commit, and deploy. The ↑ control reorders records. `featured` selects homepage entries; the first three featured published projects appear there. Change an existing ID only if you accept breaking its old shared URL.

15. Add technical articles

Choose **Articles** in the editor. The same records and sections apply; public URLs use `#/research/id`. Add tags, category, and real publication date if known. Reading time is estimated from section body word counts at 220 words/minute, rounded up with a one-minute minimum. It is an estimate, not a timed result.

Markdown supports paragraphs, lists, links, and fenced code blocks. Example:

````
```bash
node --version
```
````

Code has syntax highlighting and a copy control with feedback. Copying recursively extracts text from highlighted spans rather than serializing React objects. The article table of contents derives from section headings, not arbitrary subheadings inside Markdown. Set section images/captions in the editor; do not publish private infrastructure, credentials, personal data, or unauthorized target information. Related entries and previous/next navigation use published collection order.

16. Manage achievements

Choose **Achievements**. Preserve the supplied TechWiz facts unless you have a correction. Select the homepage recognition through `homepage.featuredAchievementId`, which matches a published achievement's stable ID. The strip reads that record's title and summary; reordering cannot attach TechWiz text to another achievement. The achievements page lists the other published records too. Add additional records only for genuine workshops, certifications, competitions, and leadership activities. No certificate, photo, or official evidence link is included yet.

17. Update the résumé

Place an approved PDF at `public/documents/mohammed-resume.pdf` (create the directory). Set `profile.resume` to `documents/mohammed-resume.pdf`. Use **Résumé** in the local editor to change this field. A public résumé is not supplied initially, so no fake download button is shown; the hero links to About instead. Check the PDF for hidden metadata, home address, private telephone numbers, and other information you do not intend to disclose.

18. Update social links and contact

Set `profile.email` to your professional email, or leave it empty. Add `profile.socials` records such as `{"label":"GitHub","url":"https://github.com/YOUR_ACCOUNT"}` only after replacing the placeholder. **Social links** edits the profile object. Empty email/social fields yield a truthful unavailable state instead of fake profiles.

With an email configured, the mailto link opens the visitor's email application; it does not send or store messages. Copy-email feedback depends on browser clipboard permissions and a secure context (HTTPS or localhost). There is no contact form backend. Social links open a new tab with `noopener noreferrer`.

19. Change colors and typography

Edit `theme.accent` and `theme.background` (six-digit hex) in content, or **Theme & settings**. Other semantic tokens are centralized at the top of `src/styles/main.css`: surface, border, text, and muted. The live SVG and card highlights now derive from the accent token; static raster sharing artwork still needs manual editing. Changing one color does not guarantee contrast; test the result. The default is dark-only; a partially implemented light theme is intentionally omitted.

Inter Variable and JetBrains Mono are local font assets from pinned Fontsource packages. Imports are in `src/main.tsx` and `src/styles/main.css`. System fallbacks remain available. Preserve font licenses under `docs/licenses/`. Update `public/images/social.png` if changing the name or brand; it is a static sharing asset, not dynamically personalized.

20. Adjust animations

Set `settings.motion` to `false` for a global animation-free experience. Motion code is in `src/components/Effects.tsx`; reveal styles, interactions, and the sphere breathing animation are in CSS. GSAP registers ScrollTrigger, scopes animations, and reverts contexts on route/unmount. Lenis runs only with motion allowed and follows the GSAP ticker; do not add another requestAnimationFrame loop. Native anchors use a `section` URL query inside the fragment; history and direct refresh are preserved. Reduced-motion media-query changes are observed live.

Hover effects are intentionally restrained. Do not hide essential text behind animation. If adding substantial effects, test mobile, keyboard focus, layout shifts, and reduced motion again.

21. Troubleshoot build errors

Run `npm run validate` first. Zod errors identify the path to a field: fix the source JSON, not generated public content. Common issues: malformed JSON, duplicate IDs in a collection, unsafe URLs, invalid status, missing required text, or date format. If an install fails, confirm Node version, network access, and unmodified lockfile, then retry `npm ci`. Avoid deleting the lockfile as a first step.

Blank deployed page: inspect the browser network tab, repository `SITE_BASE`, asset 404s, and Actions logs. Match preview/build bases. Content not changing: export the editor, replace the correct file, regenerate, rebuild, and wait for deployment. Clipboard failure: use HTTPS and manual selection. See `docs/TROUBLESHOOTING.md`.

22. Update dependencies

Work on a branch:

```
npm outdated
npm audit
npm install PACKAGE_NAME@REVIEWED_VERSION
npm run lint
npm test
npm run build
npm run check:production
```

Replace the command placeholders with a specific reviewed package/version. Review its changelog, license, Node requirements, and peer dependencies. Commit both `package.json` and `package-lock.json`. Run browser/accessibility checks and compare Lighthouse afterward. Do not run `npm audit fix --force` blindly. Exact direct versions are pinned; lockfile records transitives. No audit can prove software vulnerability-free.

23. Run tests

Portable checks:

```
npm run validate
npm run lint
npm test
npm run build
npm run check:production
npm audit
```

Vitest covers content schemas, complete HTTPS URL restrictions, calendar dates, image alt-text validation, draft visibility, search, reading time, recognition selection, highlighted-code extraction, and public component behavior. The `pretest` hook generates public content, so tests also work on a clean checkout without the ignored generated JSON. Browser harnesses are in `tests/browser-checks.cjs`, `tests/accessibility.cjs`, `tests/production.cjs`, and `tests/review-browser.cjs`. They are functions for Arena's preinstalled Playwright CLI, not Vitest tests. To rerun in the same Arena environment, start `npm run dev` on port 5173, then:

```
playwright-cli -s=arena open http://localhost:5173 --browser=chromium
playwright-cli -s=arena run-code --filename=tests/browser-checks.cjs
playwright-cli -s=arena run-code --filename=tests/accessibility.cjs
playwright-cli -s=arena close
```

The accessibility harness references `/home/user/portfolio/node_modules/axe-core/axe.min.js`; update that filesystem path outside Arena. The subpath harness expects a matching `/security-lab/` build and preview at port 4174. It is not run in the supplied GitHub Actions workflow. On a different machine, supply a compatible CLI/Chromium environment or port the function bodies to your normal Playwright runner; no browser binary is bundled in this archive.

The browser harnesses use the shipped sample titles, counts, and IDs; adapt their expected records after replacing demo content. The achievement acceptance test intentionally checks the supplied recognition. Recorded checks are under `docs/`; the delivery report states what ran. Automated accessibility scans do not replace keyboard, screen-reader, zoom, contrast, and real-device testing. Lighthouse is a lab measurement, not a guarantee of field Core Web Vitals.

24. Review security alerts

Enable GitHub dependency and secret-scanning alerts where supported. Run `npm audit` regularly and before release. The recorded audit was clean at delivery, not a permanent guarantee. Review alerts by exploitability and exposure: dev-server issues still matter if you expose a dev server. Do not publish a vulnerable dev service just because the final site is static.

25. How the admin system actually works

1. Run the development server on your own trusted machine.

2. Open `http://localhost:5173/#/control-center/`.

3. Choose a section; search records or add/edit one.

4. Save to this page's memory. A status message explicitly says it is not live.

5. Preview a record locally, including drafts. This preview does not publish it.

6. Use **Export site.json**. The browser downloads the validated whole content file.

7. Replace `src/content/site.json` with that file. Keep a backup first.

8. Run validation and build, inspect the diff, commit, and push.

9. GitHub Actions must succeed before the site changes.

There is no automatic filesystem write, repository connection, import/upload control, autosave, browser storage, account session, or background persistence. Refreshing loses unexported work. Cancel discards form edits after confirmation. Delete requires native browser confirmation. Removing sections/references and discarding changed settings also require confirmation. Unsaved work triggers a browser unload warning when the browser supports it; export still remains the only persistence mechanism. An open unsaved form is not silently included in an export: the UI warns before exporting only the previously saved working copy. Reordering changes only the working copy until exported. The editor UI is excluded from production, but its source remains in the downloadable project. The non-obvious URL is not an access control.

26. What requires a backend

Remote login, protected live writes, account roles, centrally stored drafts, shared editing, session revocation, server audit logs, and a real contact submission form require a separate trusted service. They are **not implemented or configured**. GitHub Pages supplies none of these. A React password box, localStorage flag, obscured URL, or frontend token would not fix that.

A future backend should authenticate administrators, authorize every mutation, validate and sanitize content, apply rate limits, and use an authenticated persistence layer. A GitHub integration should use a least-privileged GitHub App server-side; never send write tokens in frontend bundles.

27. Authentication and authorization

For this delivered local-only architecture: no application authentication, authorization endpoint, cookies, passwords, login, session expiration, or logout exists. Access is through your operating-system account and local development environment. Real publication is authorized by your GitHub repository permissions and Actions deployment environment. Enable GitHub MFA/passkeys and restrict maintainers. Unauthorized/expired-session tests are not applicable because there are no such server endpoints.

For a future remote editor, design and test supported authentication, backend authorization, session timeout/revocation, HttpOnly/Secure/SameSite cookies, CSRF controls where relevant, audit logging, and rate-limited login before enabling privileged operations. That is new engineering work, not an environment switch in this project.

28. Secrets outside source control

There are no required application secrets. `SITE_BASE` and public `seo.siteUrl` are not secrets. GitHub's built-in Actions identity is short-lived and configured by workflow permissions, not a manually committed token. Git authentication belongs in a credential manager/SSH agent. Any future database credentials, OAuth/GitHub App secrets, email provider keys, or signing keys belong in the trusted backend's secret manager only. A `VITE_` variable is public by design; never place secrets there.

29. Back up and recover

Keep regular encrypted/offline backups of the source JSON and public assets. Do not treat the editor's working memory as a backup. A public repository or downloadable archive is not a secure vault for private drafts. Avoid confidential material anywhere in a public Git history.

To recover, restore a known-good source JSON and matching assets, then run validation, tests, build, and production isolation checks. For an accidentally published change, revert the commit on a recovery branch, review, merge, and redeploy. Confirm the actual public artifact no longer contains the data. Removing data from the latest commit does not remove copies/history; rotate any leaked credential immediately and follow the incident steps in the checklist.

30. Maintain safely after launch

Before each publication, verify authorization, redaction, factual claims, publication rights, links, image alternative text, and résumé privacy. **Every file inside `public/` is copied to the website, even if only a draft mentions it. Keep draft-only or private screenshots/documents outside `public/` and public source control until approved for publication.** Hide samples with `settings.showSamples=false` or replace them with verified work. Keep all drafts non-sensitive even though production filtering excludes them. Never store unpublished confidential findings in the public repository.

Set `seo.siteUrl` only after the final URL is known. Canonical and sitemap are intentionally omitted/empty until then. Hash-routed views update titles and descriptions in the browser, but social scrapers usually see only the homepage metadata and fragment URLs are not separately indexed like prerendered pages. The sitemap intentionally lists only the canonical root when configured. If independently indexed article URLs and rich per-article social previews become essential, plan static prerendering or an SSG migration; do not claim hash routing solves that.

Review dependencies, workflow permissions, repository access, backup recovery, mobile behavior, and accessibility periodically. Keep the contact fallback honest if no email is configured. Re-measure performance on your actual public host and real devices after adding images or third-party code.

Reviewed release notes

The second review added a required `homepage` configuration object. If bringing content from an earlier ZIP/export, merge your profile and collections into the new `src/content/site.json` while keeping/customizing `homepage`; validate before replacing the full file. Main copy and section switches are available under Theme & settings.

Additional browser regressions are in `tests/review-browser.cjs`, using the same development server and Playwright CLI as the other browser harnesses. They cover typed comma-separated tags, exact highlighted JavaScript copying, draft-preview isolation, accessible date errors, unsaved settings

confirmation, and empty admin searches. The source editor shell and reusable record fields are separated into `src/pages/Admin.tsx` and `src/components/admin/RecordFields.tsx`.